ONLINE SHOPPING PURCHASE PREDICTION USING MACHINE LEARNING

Final Project Report

Submitted by : Upputhalla Anuradha

Institution : Rise Krishna Sai Prakasam Group of Institutions

Department : Computer Science and Engineering

CONTENTS

Abstract

1. Introduction

2. Problem Statement

3. Objectives

4. Dataset Description

5. Methodology

6. System Architecture

7. Technologies Used

8. Implementation

9. System Requirements

10. Testing

11. Results and Discussion

12. Limitations

13. Future Enhancements

14. Conclusion

References

ABSTRACT

This project presents a machine-learning-based web application that estimates whether an online shopping visitor is likely to make a purchase based on browsing-session information. The system uses session features such as the number of administrative, informational, and product-related pages viewed, time spent on those pages, bounce and exit rates, month, operating system, browser, region, traffic type, visitor type and weekend status.

The application combines a trained Python machine-learning pipeline with a FastAPI prediction service and a PHP web interface hosted through XAMPP. A user enters session details into a form, PHP sends the values to the API as JSON, and the API returns a predicted class—Purchase or No Purchase—and a purchase probability. A dashboard summarizes the dataset and displays purchase statistics and charts.

The dataset used during development contains 12,330 sessions, including 1,908 purchase sessions and 10,422 non-purchase sessions. The recorded model evaluation accuracy was approximately 61.48%. The purchase class had precision of 0.25 and recall of 0.74 in the reported evaluation. These results show that the model identified many purchase sessions, but its predictions require careful interpretation and further evaluation before real-world deployment.

1. INTRODUCTION

Online stores collect information about how visitors navigate their websites. This information can help businesses understand browsing behaviour and estimate whether a visitor may complete a purchase. A prediction system can support analysis of customer sessions and help teams explore patterns associated with purchasing.

The purpose of this project is to develop an end-to-end prototype that accepts shopping-session attributes and returns a machine-learning prediction through a simple web interface. It demonstrates the integration of data preparation, model training, API development, PHP form handling, and dashboard visualization.

2. PROBLEM STATEMENT

An online shopping session may end without a purchase or may result in a purchase. Manually inspecting large numbers of sessions is difficult. The project addresses this by using historical session data to train a classifier that estimates the likely outcome of a new session based on its features.

The output is a prediction, not a guarantee of customer behaviour. The model can make incorrect predictions, particularly when the input values are inaccurate or differ from the training data.

3. OBJECTIVES

- Explore and prepare an online shopping intention dataset.
- Train a classification model to estimate the Revenue outcome for a session.
- Process numerical and categorical features using a machine-learning pipeline.
- Save the trained model and decision threshold for use by the prediction API.
- Build a PHP form that accepts the 16 selected session features.

- Connect the PHP application to a Python FastAPI endpoint.
- Display the predicted class and estimated purchase probability.
- Create a dashboard with session totals, purchase counts, purchase rate, and charts.

4. DATASET DESCRIPTION

The project uses an online shoppers purchasing-intention dataset with 12,330 session records. The target variable is Revenue, which indicates whether the session ended in a purchase. The class distribution recorded during project development was 10,422 False (no purchase) and 1,908 True (purchase). This means that purchase sessions make up approximately 15.47% of the dataset, so the target classes are imbalanced.

The model uses 16 input features. The Revenue target is not an input feature. PageValues was also excluded from the selected inputs in the project implementation.

4.1 Input Features

Administrative — number of administrative pages viewed.

Administrative_Duration — time spent on administrative pages.

Informational — number of informational pages viewed.

Informational_Duration — time spent on informational pages.

ProductRelated — number of product-related pages viewed.

ProductRelated_Duration — time spent on product-related pages.

BounceRates — bounce-rate measure for the session.

ExitRates — exit-rate measure for the session.

SpecialDay — closeness to a special day or event, represented numerically.

Month — month associated with the session.

OperatingSystems — operating-system category code.

Browser — browser category code.

Region — region category code.

TrafficType — traffic-source category code.

VisitorType — visitor category, such as returning or new visitor.

Weekend — whether the session occurred on a weekend.

5. METHODOLOGY

The project workflow consists of dataset exploration, feature selection, preprocessing, model training, evaluation, saving the model artefacts, API development, PHP integration, and dashboard presentation.

5.1 Data Preparation and Preprocessing

The selected inputs contain numerical and categorical data. Numerical features are scaled using StandardScaler, while categorical features are encoded using OneHotEncoder as part of the machine-learning pipeline. Keeping preprocessing inside the pipeline ensures that the same transformations are applied during training and prediction. The target Revenue is used for training and evaluation, not as a user input.

5.2 Model Training and Evaluation

A Random Forest-based classification pipeline was used in the project. The trained pipeline and a separate decision threshold were saved as files named purchase_prediction_model.pkl and purchase_threshold.pkl. The API loads these artefacts to make predictions on new session inputs.

The project's recorded evaluation accuracy was 0.6148 (61.48%). In the reported classification results, the No Purchase class had precision 0.92, recall 0.59, and F1-score 0.72 on 2,084 evaluation samples. The Purchase class had precision 0.25, recall 0.74, and F1-score 0.37 on 382 evaluation samples. The supports sum to 2,466 evaluation samples.

Accuracy alone is not sufficient for an imbalanced dataset. The purchase-class recall of 0.74 indicates that the model identified a substantial portion of actual purchase sessions in that evaluation, while the purchase precision of 0.25 indicates that many sessions predicted as purchases were not actual purchases. These metrics should be reported honestly and should not be interpreted as production-level performance.

6. SYSTEM ARCHITECTURE

The application uses three main layers:

1. Presentation layer — index.php displays a styled form with the 16 selected input fields. dashboard.php reads the dataset and presents summary statistics and charts.

2. PHP integration layer — predict.php validates that the expected fields were submitted, converts values to suitable types, sends the input data as JSON to the Python API, and displays the response.

3. Machine-learning service — api.py is a FastAPI application. Its POST /predict endpoint converts the incoming session into a pandas DataFrame, selects the model's expected feature order, calculates the purchase probability, applies the saved threshold, and returns the prediction and probability.

7. TECHNOLOGIES USED

Python — data processing and model training.

pandas — tabular data handling.

scikit-learn — preprocessing and classification pipeline.

joblib — saving and loading the trained model artefacts.

FastAPI — exposing the model through an HTTP API.

Uvicorn — running the FastAPI development server.

PHP — handling the web form and communicating with the API.

HTML and CSS — building and styling the web pages.

Chart.js — rendering dashboard charts in the browser.

XAMPP / Apache — running the PHP application locally.

Jupyter Notebook — interactive data exploration and model development.

8. IMPLEMENTATION

8.1 Model artefacts: The training workflow saves the trained pipeline as purchase_prediction_model.pkl and the selected threshold as purchase_threshold.pkl. Both files must be available in the project directory from which api.py loads them.

8.2 API: The FastAPI service defines a ShoppingSession schema with all 16 features. The /predict endpoint receives JSON data, prepares it as a DataFrame, orders the columns according to model.feature_names_in_, calculates predict_proba, and compares the probability with the saved threshold. The response includes prediction and purchase_probability.

8.3 PHP form: index.php presents inputs for page activity, session rates, month, and visitor information. Required fields and dropdown prompts help prevent accidental empty submissions. The field names must match the API schema.

8.4 PHP-to-API connection: predict.php receives the form submission and sends a JSON POST request to http://127.0.0.1:8000/predict. The API must be running for this request to succeed. The result page shows the predicted class and the probability as a percentage.

8.5 Dashboard: dashboard.php reads the CSV dataset, counts sessions, purchases, and non-purchases, calculates the purchase rate, and presents the summary. Chart.js is used for a doughnut chart comparing purchase and non-purchase sessions and a bar chart of monthly purchases, provided the Chart.js library loads successfully.

9. SYSTEM REQUIREMENTS

Hardware: A standard laptop or desktop computer capable of running Python, a web browser, and XAMPP.

Software: Windows or another compatible operating system; Python; the project's Python dependencies; VS Code or another code editor; XAMPP with Apache; a modern web browser; and the dataset CSV.

Network/runtime note: The prototype uses a local API address (127.0.0.1:8000) and is intended for local demonstration. The API terminal must remain running while predictions are made. If Chart.js is loaded from a CDN, an internet connection may be needed for the charts.

10. TESTING

Functional testing should cover the following cases:

- Open the dashboard and verify the dataset totals and charts.
- Open index.php and verify that all 16 input fields appear.
- Confirm that visitor type and weekend dropdowns initially display their placeholder prompts.
- Attempt to submit without completing required fields and verify browser validation.
- Submit a valid session and confirm that predict.php communicates with the API.
- Verify that the result page displays Purchase or No Purchase and the purchase probability.
- Stop the API and confirm that the application displays a helpful connection error rather than silently failing.
- Try multiple plausible input combinations and check that the page remains functional.

Testing status: The project conversation records successful API testing through the FastAPI documentation interface and successful loading of the PHP dashboard. The final styled form and end-to-end path should be tested again on the submission machine before claiming every test has passed.

11. RESULTS AND DISCUSSION

The prototype demonstrates an end-to-end flow from user-entered session information to a machine-learning result. It combines a model pipeline with a local web application and provides a dashboard for exploring dataset-level purchase patterns.

The model's recorded accuracy is 61.48%. Because the dataset is imbalanced and purchase precision is relatively low, the prediction should be treated as an estimate for educational demonstration rather than a definitive decision. A probability such as 0.5549 means the model assigned an estimated purchase probability of 55.49% to that input under the trained model; it does not mean that a purchase is certain.

12. LIMITATIONS

- The model is trained on a historical dataset and may not generalize to all online stores or current visitor behaviour.
- Purchase sessions are the minority class, which complicates evaluation.
- The reported purchase precision indicates that many predicted purchases may be false positives.
- User-entered values can be inaccurate or inconsistent with the original data distribution.
- The local API must be running for PHP predictions to work.
- The prototype is not configured as a secure, production deployment and does not include user authentication, monitoring, or production hosting.

13. FUTURE ENHANCEMENTS

- Compare multiple algorithms and tune hyperparameters using cross-validation.
- Evaluate precision-recall curves, ROC-AUC, confusion matrix, calibration, and class-specific metrics.
- Revisit the decision threshold based on the business cost of false positives and false negatives.
- Add stronger server-side validation, clearer error handling, and automated tests.
- Connect the form to real session data instead of requiring manual entry.
- Improve dashboard filtering, accessibility, and chart presentation.
- Deploy the API and PHP application in a properly secured hosting environment if required.

14. CONCLUSION

The Online Shopping Purchase Prediction project demonstrates how machine learning can be integrated into a web application. A Python classification pipeline processes browsing-session features and estimates the likelihood of a purchase. FastAPI exposes the model, PHP connects the user interface to the API, and the dashboard summarizes historical session data.

The prototype meets its educational goal of demonstrating data preparation, model training, API integration, and web development. The recorded model results also highlight the importance of evaluating precision and recall—not only accuracy—when purchase outcomes are imbalanced. Further validation and model improvement would be necessary before using the system for real business decisions.

REFERENCES

1. Online Shoppers Purchasing Intention dataset, UCI Machine Learning Repository.

2. scikit-learn documentation: preprocessing, model evaluation, and Random Forest classification.

3. FastAPI documentation: request bodies and path operations.

4. PHP documentation: forms, superglobals, JSON encoding, and HTTP stream contexts.

5. Chart.js documentation: doughnut and bar charts.